WEEK 7: SQL QUERY IMPLEMENTATION FOR SAREE BOUTIQUE DATABASE

SCENARIO

The Saree Boutique maintains database records related to saree products, product categories, inventory, customer orders, payments, and product reviews. The management team wants to retrieve useful information from the database for daily operations and basic business analysis.

SQL queries are used to search, filter, sort, and generate reports from the existing Saree Boutique Database.

TASKS TO BE DONE

1. PERFORM BASIC SQL QUERIES

Concept

The SQL SELECT command is used to retrieve data from database tables.

Task 1.1: Display all available saree products.

```
SQL> SELECT *
  2  FROM PRODUCT1;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P101
Shoe
CAT101         378             40

P111
Sketch
CAT101          28             10

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------

P102
T-Shirt
CAT102         599             25

P001
Kanchipuram Silk Saree

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
SC001         8500             10

P002
Cotton Saree
 SC002        1800             25


SQL>
```

Task 1.2: Display all available inventory records.

```
SQL> SELECT * FROM PRODUCT1;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P101
Shoe
CAT101         378             40

P111
Sketch
CAT101          28             10

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------

P102
```

Task 1.3: Retrieve product names and prices only.

```
SQL> SELECT * FROM PRODUCT1;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P101
Shoe
CAT101         378             40

P111
Sketch
CAT101          28             10

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------

P102
T-Shirt
CAT102         599             25

P001
Kanchipuram Silk Saree

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
SC001         8500             10

P002
Cotton Saree
 SC002        1800             25
```

Task 1.4: Display all orders placed by customers.

```
SQL> SELECT * FROM ORDERS;

ORDER_I CUSTOME ORDER_DATE          TOTAL_AMOUNT ORDER_STATUS
------- ------- ------------------- ------------ --------------------
CUSTOMER_NAME
--------------------------------------------------
ORD0001 CUS0001 24-SEP-26                   25000 Pending


ORD0002 CUS0002 24-SEP-26                   15000 Shipped
```

Task 1.5: Retrieve the different payment modes.

```
SQL> SELECT * FROM PAYMENT;

PAYMENT_ID ORDER_I PAYMENT_DATE       PAYMENT_MODE         PAYMENT_STATUS
---------- ------- ------------------ -------------------- --------------------
TRANSACTION_AMOUNT
------------------
P001       ORD0001 01-SEP-26          UPI                  Success
              1500

P002       ORD0002 02-SEP-26          Card                 Success
              2500

P003       ORD0002 03-SEP-26          UPI                  Pending
              1800


PAYMENT_ID ORDER_I PAYMENT_DATE       PAYMENT_MODE         PAYMENT_STATUS
---------- ------- ------------------ -------------------- --------------------
TRANSACTION_AMOUNT
------------------
P004       ORD0002 04-SEP-26          Cash                 Success
              1200

P005       ORD0001 05-SEP-26          UPI                  Success
               900
```

2. APPLY FILTERING CONDITIONS USING WHERE

Concept

The WHERE clause filters records based on specified conditions.

Task 2.1: Find saree products with a price greater than ₹5,000.

```
SQL> SELECT * FROM PRODUCT1 WHERE PRICE > 5000;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P001
Kanchipuram Silk Saree
SC001         8500             10


SQL> SELECT * FROM PRODUCT1 WHERE STOCK_QUANTITY > 0;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P101
Shoe
CAT101         378             40

P111
Sketch
CAT101          28             10

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------

P102
T-Shirt
CAT102         599             25

P001
Kanchipuram Silk Saree

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
SC001         8500             10

P002
Cotton Saree
 SC002        1800             25
```

Task 2.2: Display products currently available in inventory.

```
SQL> SELECT * FROM INVENTORY WHERE STOCK_STATUS = 'Available';

INVENTORY_ID PRODUCT STOCK_QUANTITY STOCK_STATUS          SELLER_ID
------------ ------- -------------- -------------------- ----------
           1 P001                10 Available                     1
           2 P002                15 Available                     2
           4 P002                 8 Available                     4
           3 P001                20 Available                     3

SQL>
```

Task 2.3: Display completed orders.

```
SQL> SELECT * FROM ORDERS WHERE ORDER_STATUS = 'Completed'

no rows selected
```

Task 2.4: Find products with a rating above 4.

```
SQL> SELECT * FROM SAREE_REVIEW WHERE RATING > 4;

no rows selected
```

3. SORT DATA USING ORDER BY

Concept

The ORDER BY clause sorts records in ascending or descending order.

Task 3.1: Display products from lowest to highest price.

```
SQL> SELECT * FROM PRODUCT1 ORDER BY PRICE ASC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P111
Sketch
CAT101          28             10

P101
Shoe
CAT101         378             40

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------

P102
T-Shirt
CAT102         599             25

P002
Cotton Saree

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
 SC002        1800             25

P001
Kanchipuram Silk Saree
SC001         8500             10
```

Task 3.2: Display customer names alphabetically using order records.

```
SQL> SELECT CUSTOMER_ID, CUSTOMER_NAME FROM ORDERS ORDER BY CUSTOMER_NAME ASC;

CUSTOME CUSTOMER_NAME
------- --------------------------------------------------
CUS0002 NANDY
CUS0001 YAMINI
```

Task 3.3: Find the most expensive saree products.

```
SQL> SELECT * FROM PRODUCT1 ORDER BY PRICE DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P001
Kanchipuram Silk Saree
SC001         8500             10

P002
Cotton Saree
 SC002        1800             25

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------

P102
T-Shirt
CAT102         599             25

P101
Shoe

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
CAT101         378             40

P111
Sketch
CAT101          28             10
```

Task 3.4: Display the latest orders first.

```
SQL> SELECT * FROM ORDERS ORDER BY ORDER_DATE DESC;

ORDER_I CUSTOME ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
------- ------- ------------------ ------------ ------------------
CUSTOMER_NAME
--------------------------------------------------
ORD0002 CUS0002 24-SEP-26                 15000 Shipped
NANDY

ORD0001 CUS0001 24-SEP-26                 25000 Pending
YAMINI
```

4. RETRIEVE UNIQUE VALUES USING DISTINCT

Concept

The DISTINCT keyword retrieves unique values and removes duplicates from the query result.

Task 4.1: Display unique saree categories.

```
SQL> SELECT DISTINCT CATEGORY_ID FROM PRODUCT1;

CATEGOR
-------
CAT101
SC001
 SC002
CAT102
```

Task 4.2: Display different payment modes used for transactions.

```
SQL> SELECT DISTINCT PAYMENT_MODE FROM PAYMENT;

PAYMENT_MODE
--------------------
Card
Cash
UPI
```

5. SEARCH PRODUCTS BASED ON CONDITIONS

Concept

SQL conditions such as BETWEEN, LIKE, and comparison operators are used to search for products that satisfy particular requirements.

Task 5.1: Find products within the price range ₹20,000 to ₹40,000.

```
SQL> SELECT * FROM PRODUCT1
  2  WHERE PRICE BETWEEN 20000 AND 40000;

no rows selected
```

Task 5.2: Display products belonging to a particular saree category.

```
SQL> SELECT * FROM PRODUCT1
  2  WHERE CATEGORY_ID = 'SC001';

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
----------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P001
Kanchipuram Silk Saree
SC001         8500             10
```

Task 5.3: Find products currently available in inventory.

```
SQL> SELECT * FROM INVENTORY;

INVENTORY_ID PRODUCT STOCK_QUANTITY STOCK_STATUS          SELLER_ID
------------ ------- -------------- -------------------- ----------
           1 P001                10 Available                     1
           2 P002                15 Available                     2
           4 P002                 8 Available                     4
           3 P001                20 Available                     3

SQL> SELECT * FROM INVENTORY
  2  WHERE STOCK_STATUS = 'Available';

INVENTORY_ID PRODUCT STOCK_QUANTITY STOCK_STATUS          SELLER_ID
------------ ------- -------------- -------------------- ----------
           1 P001                10 Available                     1
           2 P002                15 Available                     2
           4 P002                 8 Available                     4
           3 P001                20 Available                     3
```

Task 5.4: Search for products using the product name.

```
SQL> SELECT * FROM PRODUCT1
  2  WHERE PRODUCT_NAME LIKE '%Saree%';

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
----------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P001
Kanchipuram Silk Saree
SC001         8500             10

P002
Cotton Saree
 SC002        1800             25

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
----------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
```

Task 5.5: Find low-stock products with a quantity below 30.

```
SQL> SELECT * FROM INVENTORY
  2  WHERE STOCK_QUANTITY < 30;

INVENTORY_ID PRODUCT STOCK_QUANTITY STOCK_STATUS          SELLER_ID
------------ ------- -------------- -------------------- ----------
           1 P001                10 Available                     1
           2 P002                15 Available                     2
           4 P002                 8 Available                     4
           3 P001                20 Available                     3
```

6. RETRIEVE CUSTOMER AND PRODUCT INFORMATION

Concept

SQL joins combine related information from multiple tables using common columns.

Task 6.1: Display customer details along with their orders.

```
SQL> SELECT CUSTOMER_ID,
  2         CUSTOMER_NAME,
  3         ORDER_ID,
  4         ORDER_DATE,
  5         TOTAL_AMOUNT,
  6         ORDER_STATUS
  7  FROM ORDERS
  8  ORDER BY CUSTOMER_ID, ORDER_DATE;

CUSTOME CUSTOMER_NAME                                      ORDER_I
------- -------------------------------------------------- -------
ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
------------------ ------------ --------------------
CUS0001 YAMINI                                             ORD0001
24-SEP-26                 25000 Pending

CUS0002 NANDY                                              ORD0002
24-SEP-26                 15000 Shipped
```

Task 6.2: Display product details with category information.

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      P.PRICE,
  5      P.STOCK_QUANTITY,
  6      C.CATEGORY_ID,
  7      C.CATEGORY_NAME,
  8      C.DESCRIPTION
  9  FROM PRODUCT1 P
 10  JOIN CATEGORY1 C
 11      ON P.CATEGORY_ID = C.CATEGORY_ID
 12  ORDER BY C.CATEGORY_NAME, P.PRODUCT_NAME;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY CATEGOR CATEGORY_NAME
---------- -------------- ------- --------------------
DESCRIPTION
--------------------------------------------------
P002
Cotton Saree
      1800             25  SC002  Cotton Sarees
 Comfortable cotton sarees


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY CATEGOR CATEGORY_NAME
---------- -------------- ------- --------------------
DESCRIPTION
--------------------------------------------------
P001
Kanchipuram Silk Saree
      8500             10 SC001   Kanchipuram Silk
 Traditional silk sarees


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY CATEGOR CATEGORY_NAME
---------- -------------- ------- --------------------
DESCRIPTION
--------------------------------------------------
P001
Kanchipuram Silk Saree
      8500             10 SC001   Kanchipuram Silk
Traditional silk sarees


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY CATEGOR CATEGORY_NAME
---------- -------------- ------- --------------------
DESCRIPTION
--------------------------------------------------
P001
Kanchipuram Silk Saree
      8500             10 SC001   Kanchipuram Silk
Traditional silk sarees
```

Task 6.3: Find customers who purchased a specific saree product.

```
SQL> SELECT DISTINCT
  2      O.CUSTOMER_ID,
  3      O.CUSTOMER_NAME,
  4      OD.PRODUCT_ID
  5  FROM ORDERS O
  6  JOIN ORDER_DETAILS OD
  7      ON O.ORDER_ID = OD.ORDER_ID
  8  WHERE OD.PRODUCT_ID = 'P001';

CUSTOME CUSTOMER_NAME                                      PRODUCT
------- -------------------------------------------------- -------
CUS0002 NANDY                                              P001
```

Task 6.4: Display products purchased by each customer.

```
SQL> SELECT
  2      O.CUSTOMER_ID,
  3      O.CUSTOMER_NAME,
  4      P.PRODUCT_ID,
  5      P.PRODUCT_NAME,
  6      OD.QUANTITY
  7  FROM ORDERS O
  8  JOIN ORDER_DETAILS OD
  9      ON O.ORDER_ID = OD.ORDER_ID
 10  JOIN PRODUCT1 P
 11      ON OD.PRODUCT_ID = P.PRODUCT_ID
 12  ORDER BY O.CUSTOMER_ID;

CUSTOME CUSTOMER_NAME                                      PRODUCT
------- -------------------------------------------------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------
CUS0001 YAMINI                                             P111
Sketch
         1

CUS0001 YAMINI                                             P101
Shoe
         2

CUSTOME CUSTOMER_NAME                                      PRODUCT
------- -------------------------------------------------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------

CUS0002 NANDY                                              P001
Kanchipuram Silk Saree
         1

CUS0002 NANDY                                              P002
Cotton Saree

CUSTOME CUSTOMER_NAME                                      PRODUCT
------- -------------------------------------------------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------
         2

CUS0002 NANDY                                              P102
T-Shirt
         2
```

7. APPLY MULTIPLE FILTERING CONDITIONS

Concept

Multiple filtering conditions help retrieve specific records using comparison operators, LIKE, and date conditions.

Task 7.1: Find saree products costing more than ₹5,000.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         P.PRICE,
  4         C.CATEGORY_NAME
  5  FROM PRODUCT1 P
  6  JOIN CATEGORY1 C
  7      ON P.CATEGORY_ID = C.CATEGORY_ID
  8  WHERE P.PRICE > 10000;

no rows selected
```

Task 7.2: Search for products containing the word “Saree”.

```
SQL> SELECT *
  2  FROM PRODUCT1
  3  WHERE LOWER(PRODUCT_NAME) LIKE '%saree%';

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
P001
Kanchipuram Silk Saree
SC001         8500             10

P002
Cotton Saree
 SC002        1800             25

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY
------- ---------- --------------
```

Task 7.3: Find orders placed within a specific date range.

```
SQL> SELECT *
  2  FROM ORDERS
  3  WHERE ORDER_DATE BETWEEN
  4        TO_DATE('01-09-2026', 'DD-MM-YYYY')
  5    AND TO_DATE('30-09-2026', 'DD-MM-YYYY');

ORDER_I CUSTOME ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
------- ------- ------------------ ------------ ---------------
CUSTOMER_NAME
--------------------------------------------------
ORD0001 CUS0001 24-SEP-26                 25000 Pending
YAMINI

ORD0002 CUS0002 24-SEP-26                 15000 Shipped
NANDY
```

8. GENERATE BASIC BUSINESS REPORTS

Report 1: Product Availability Report

Concept

A product availability report displays product names, prices, stock quantities, and availability status by joining product and inventory information.

```
SQL> SELECT P.PRODUCT_NAME,
  2         P.PRICE,
  3         I.STOCK_QUANTITY,
  4         I.STOCK_STATUS
  5  FROM PRODUCT1 P
  6  JOIN INVENTORY I
  7      ON P.PRODUCT_ID = I.PRODUCT_ID;

PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY STOCK_STATUS
---------- -------------- --------------------
Kanchipuram Silk Saree
      8500             20 Available

Kanchipuram Silk Saree
      8500             10 Available

Cotton Saree
      1800              8 Available


PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY STOCK_STATUS
---------- -------------- --------------------
Cotton Saree
      1800             15 Available
```

Report 2: Customer Report

Concept

A customer report summarizes the distinct customers recorded in order transactions.

```
SQL> SELECT COUNT(DISTINCT CUSTOMER_ID) AS TOTAL_CUSTOMERS
  2  FROM ORDERS;

TOTAL_CUSTOMERS
---------------
              2
```

New Customer Registrations: Not Required — the current database schema does not contain a separate customer registration table or registration date.

Report 3: Order Report

Concept

An order report summarizes the total number of orders and the number of orders in each specified status.

```
SQL> SELECT
  2      COUNT(*) AS TOTAL_ORDERS,
  3      SUM(CASE WHEN UPPER(ORDER_STATUS) = 'COMPLETED'
  4          THEN 1 ELSE 0 END) AS COMPLETED_ORDERS,
  5      SUM(CASE WHEN UPPER(ORDER_STATUS) = 'PENDING'
  6          THEN 1 ELSE 0 END) AS PENDING_ORDERS,
  7      SUM(CASE WHEN UPPER(ORDER_STATUS) = 'CANCELLED'
  8          THEN 1 ELSE 0 END) AS CANCELLED_ORDERS
  9  FROM ORDERS;

TOTAL_ORDERS COMPLETED_ORDERS PENDING_ORDERS CANCELLED_ORDERS
------------ ---------------- -------------- ----------------
           2                0              1                0
```

Report 4: Product Performance Report

Concept

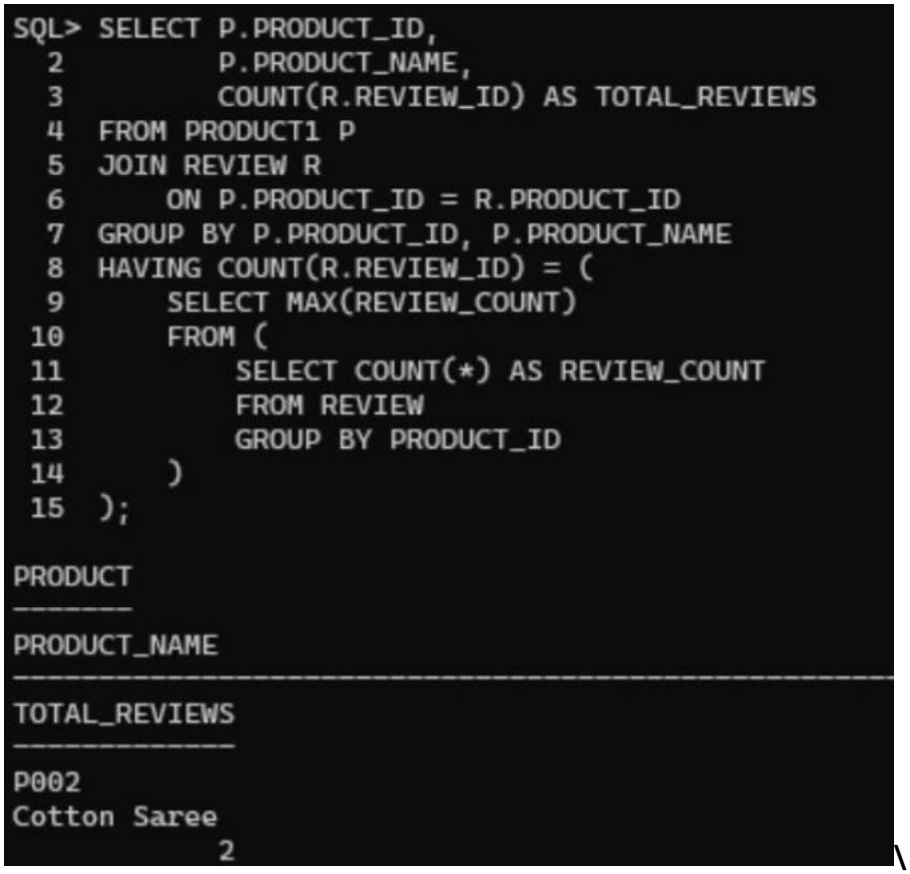
A product performance report identifies the highest-priced products, the most reviewed products, and products currently available in inventory.

Task 4.1: Find the highest-priced product or products.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM PRODUCT1
  3  WHERE PRICE = (SELECT MAX(PRICE) FROM PRODUCT1);

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
P001
Kanchipuram Silk Saree
      8500
```

Task 4.2: Find the most reviewed product or products.



```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS
  4  FROM PRODUCT1 P
  5  JOIN REVIEW R
  6      ON P.PRODUCT_ID = R.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  HAVING COUNT(R.REVIEW_ID) = (
  9      SELECT MAX(REVIEW_COUNT)
 10      FROM (
 11          SELECT COUNT(*) AS REVIEW_COUNT
 12          FROM REVIEW
 13          GROUP BY PRODUCT_ID
 14      )
 15  );

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------
TOTAL_REVIEWS
-------------
P002
Cotton Saree
            2
```

\

Task 4.3: Display products currently available in inventory.



CONCLUSION

The SQL queries were designed for the Saree Boutique Database to retrieve product and inventory details, filter and sort records, combine related information using joins, and generate basic business reports. These queries support routine database operations and help summarize the available product, order, payment, and review information.